Exercise 1.26 Explanation

Base (expmod a n n)

Let's say you have a given n. If you wanted to manipulate n to change the amount of times expmod is called, and you wanted to force expmod to be called one additional time, would need to double n in order for that to happen. Why? Remember that expmod halves n on every other call. Therefore, you need n to be twice as big for another call to expmod. Imagine a parent node in a tree. The parent node above (expmod a n n) is (expmod a 2n n), since n is halved on every other call to expmod. Since you need to double n to generate a parent node in the expmod tree, that means that expmod takes $\log_2(n)$ time.

Louis Reasoner's version

Louis Reasoner's version of expmod uses:
(* (expmod a (/ n 2) n) (expmod a (/ n 2) n))

instead of:
(square (expmod a n n)) when n is even.

This means that every other call to expmod will generate two calls to expmod, instead of just one. This means that whenever you add another parent node of expmod, that will actually add two child nodes of expmod in the call tree. However, as we discussed in the first section, you need n to double before a parent node can be added to the tree, due to expmod being $\log_2(n)$ time. **Then, it can be said that you will double the size of the tree every time that n is doubled. Double the size of the computations every time you double the size of the input. That is linear time, O(n).**